

Introduction to Embedded Linux

M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023



Institut Mines-Télécom

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

Pedagogic Objectives

- Being able to implement a Linux system on an embedded system
- Understanding of boot mechanism -from the the bootloader to the applications
- Understanding the different elements needed and how to generate them

  (kernel, device tree, initial file system, root file system...)
- Know how to write a simple device driver

- Examples of device drivers????

Pre-requisites

- Programming in C language –for writing device drivers
- Basic use of Linux, the command line. Classic compilation tools

  (gcc, make…)
- Architecture of computers and embedded systems

  (buses, interrupts, DMA, virtual memory)
- Basic use of git

Timetable 2022-2023

- Tuesday 13h30-17h 13/12 (1 Session)
- Tuesday 13h30-17h 03/01 (1 Session)
- Tuesday 13h30-17h 10/01 (1 Session)
- Tuesday 13h30-17h 17/01 (1 Session)
- Tuesday 13h30-17h 24/01 (1 Session)
- Tuesday 13h30-17h 31/01 (1 Session)
- Tuesday 13h30-17h 07/02 (1 Session)
- Tuesday 13h30-17h 14/02 (1 Session)
- Tuesday 13h30-17h 20/02 (Exam)

Practical Work

- Themes
  - Compiling the Kernel
  - Creating file system (distribution)
  - Creating a device driver
- Use of Qemu to simulate an embedded system based on ARM Cortex-A processor
  - No need for TP mock-ups (work at home, remote access)
  - Easier to debug

Practical Work (Personal Computer)

- Computer under Linux with a relatively recent distribution
- 8 GB of RAM
- 50 GB of free disk space (kernel sources, disk image, etc.)
- Classic development tools (gcc, make, git...)
- Other necessary software (qemu, arm-linux-eabi-gcc...) will be installed afterwards

Action Plan

- **Session 1**

  - Course Introduction

  - Course Starting a Linux system (beginning)

- **Session 2**

  - Course Starting a Linux system (end)

  - TP Kernel, init, initramfs, BusyBox and U-Boot

- **Session 3**

  - Course Root file system

  - TP Build your complete distribution with Buildroot

Action Plan

- **Session 4**

  - Course introduction to kernel development,
  - first module, debugging mechanisms and memory allocation
  - TP Write your first module

- **Session 5**

  - Course Device Model
  - TP

- **Session 6**

  - Course Interface with user space
  - Course Scheduling and waiting

Action Plan

- **Session 7**

  - Course Interruptions

  - TP

  Session 8

  - Course Concurrence

  - Course Access to memory mapped devices memory

  - TP

Evaluation Criteria

- **Final grade**

- - 25% of the TP renderings on the device driver (Group Assignment)
- - 75% final exam on table

  - MCQ type questions and/or open questions

  - Only document allowed: one A4 page with what you want to write on it

  - Any other document, course material, computer telephone, calculator... forbidden

  - For the moment, the conditions are met to take this exam in person. Nevertheless, if the situation requires it, this exam could take place at a distance with adapted modalities

Some Remarks!!

- Course taken over from last year (taught by another teacher in previous years) and reassembled from scratch in English
- Medium of course will remain English
- Do not hesitate to point out any error that you find in the in the course, the texts of the practical exercises, etc.
- Your feedback will be precious to improve this course for the next years
- Don't hesitate to ask questions during the course and the tutorials!

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

The world of embedded systems

- Core: from a small 8-bit microcontroller to several 64-bit multi-core/multi-threaded 64-bit processors
- RAM: from a few KB to several GB
- Mass storage: from a few KB of ROM to several GB/TB of flash or hard disk
- Memory protection: MPU (Memory Protection Unit), MMU (Memory Management Unit)
- Constraints: cost, consumption, real time, safety, security, etc.
- We will see later on the minimal characteristics to run Linux

The role of an operating system (OS)

- Securely share the resources of the execution media between the different tasks
  - Processor(s): allocates processor time to the different tasks wishing to execute (with priority management, preemption, etc.)
  - Memory: distributes the memory to the tasks and ensures isolation between them (based on hardware: MPU, MMT, etc.)
  - Peripherals
- Provides
  - Hardware abstraction for tasks
  - Communication and synchronization mechanisms between tasks

Operating Systems in embedded systems

Examples

- Generalists : Linux, *BSD (FreeBSD, NetBSD...), Windows...
- Dedicated
  - Windows IoT (ex. Windows Embedded, proprietary), QNX (proprietary), iOS (proprietary), Android (open source)...
  - Real time: VxWorks (proprietary), FreeRTOS (open source), ChibiOS/RT (open source), INTEGRITY (proprietary, classified), RTX (proprietary)...

Operating Systems in embedded systems (Cont…)

- According to [1], operating systems used in embedded systems:
  - Embedded Linux: 21%
  - Home: 19%
  - FreeRTOS: 18%
  - Ubuntu: 14%
  - Debian: 13% of the total
  - Android: 13% of the total
  - Windows 10: 10% of the total
    - Multiple answers allowed, sum > 100%.
- The market share of Linux (all distributions combined) is therefore very important in the embedded world

Why chose Linux?

- Free license and open source code
- Support for many architectures
- Many device drivers
- Dynamic ecosystem
- Long-term viability

Why not to chose Linux?

- Free license and open source code
- No commercial support
- Liability in case of problems
- Large ecosystem
- Not suitable for all embedded systems

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

Free Lisences?

Rights and Obligations

- Important point to check, especially if you distribute or sell an embedded system
- containing free software
- Some licenses impose constraints on access to the source code, patents or the possibility to update the software
- Main licenses used
  - Copyleft licenses
    - GNU General Public License (GPL) v2
    - GNU General Public License (GPL) v3
    - GNU Lesser General Public License (LGPL)
  - Permissive licenses
    - BSD, MIT, Apache

GNU General Public License (GPL) V2 [3]

- Used by the Linux kernel, many tools (busybox for example) and some libraries (GNU Readline!)
- Rights (for the user)
  - Use the software
  - Study how it works
  - Copy and distribute the software
  - Modify the software and distribute those modifications

GNU General Public License (GPL) V2 [3]

- Obligations (when distributing the software)
  - Keep the copyright notices
  - Provide access to the source code
  - Modifications made to the software must be distributed under the same terms (and therefore distribute the source code of the modifications)
- The link with a static or dynamic library is considered a derivative work and therefore requires publication of the software under the GPL
- A module compiled outside of the kernel source tree can be distributed under another license (but it will not have access to certain features of the kernel, and the kernel will be marked as tainted)

GNU General Public License (GPL) V3 [4]

- Main additions to v2
  - In case of hardware + software distribution, obligation to provide the necessary information so that the user can run a modified version of the software
  - In case of distribution of the software, provision of a license on the patents necessary to the user to exercise his rights
  - In case of an attack for patent infringement the attacker loses the rights granted by the license

GNU Lesser General Public License (LGPL) [5]

- Similar to the GPL, especially dedicated to libraries
- Facilitates the use of an LGPL library by a non-GPL/LGPL application
  - In case of static linking, it is necessary to allow the user to modify the LGPL library and link it back to your application (so provide your application in object form for example)
  - In case of dynamic linking, if the LGPL library is already present on the user's computer, there is no need to distribute the code of the library.

Permissive licenses (BSD, MIT, Apache...)

- Unlike copyleft licenses, they do not require the distribution of derivative works under the same license
- In general, in case of redistribution, they oblige simply mention the use of the component, its license and its authors, and the absence of warranty

Free licenses (Conclusion)

- Be very careful with licenses: if you have any doubt, ask a specialized lawyer
- Important point: almost all free licenses explicitly mention the absence of guarantees on the functioning of the software component
  - You will not be able to turn against such a software component if a bug in the latter has caused a critical failure of your system.
  - The legality of this exclusion is debated in some countries

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

The Linux Kernel –Brief History

- In 1991, while studying in Helsinki (Finland), Linus Torvalds (21 years old), started to write a small kernel intended to run on a 80386 processor
- On August 25th 1991, he announced his project on the newsgroup comp.os.minix

```
Hello everybody out there using minix -

I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu)
for 386(486) AT clones. This has been brewing since april, and is starting to get ready. I'd
like any feedback on things people like/dislike in minix, as my OS resembles it somewhat
(same physical layout of the file-system (due to practical reasons) among other things).

I've currently ported bash(1.08) and gcc(1.40), and things seem to work. This implies that
I'll get something practical within a few months, and I'd like to know what features most
people would want. Any suggestions are welcome, but I won't promise I'll implement them :-)

Linus (torvalds@kruuna.helsinki.fi)

PS. Yes - it's free of any minix code, and it has a multi-threaded fs. It is NOT protabl
(uses 386 task switching etc), and it probably never will support anything other than
AT-harddisks, as that's all I have :-(.
```

The Linux Kernel –Brief History

- September 1991 : version 0.01 published on a FTP server, about 10,000 lines of code
- December 1992: version 0.99 distributed under GNU GPL license
- Thanks to the democratization of the Internet in the early 90's, Linux federates a growing community of developers
- March 1994: Version 1.0.0 (175,000 lines of code)
- January 1999: Version 2.2.0 (1.800.000 lines of lines of code)
- January 2001: Version 2.4.0 (3,400,000 lines of lines of code)
- December 2003: Version 2.6.0 (6,000,000 lines of code)

The Linux Kernel –Minimum equipment required

- 32-bit or higher processor (see Embedded Linux Kernel Subset [2] for minimal support for 16-bit processors)
  - Supported architectures (v5.9): DEC Alpha, ARC, ARM, ARM 64, C6x, C-SKY, Renesas H8, Qualcomm Hexagon, Intel IA-64, Freescale 68k, Xilinx Microblaze, MIPS, Andes NDS32, Altera NiosII, OpenRISC, HP PA-RISC, PowerPC, RISC-V, IBM System/390, SuperH, SPARC, x86, x86-64, Tensilica Xtensa
- Memory Management Unit (MMU) (see µClinux for systems without MMU, it supports microcontrollers)
- Several MB of RAM (the minimum depends on the kernel configuration, around 4-8 MB, but then you have to count the executables)

The Linux Kernel

- It is not real time
  - Patch PREEMPT_RT to make the kernel able to respect hard real-time constraints
  - RTLinux : real time microkernel running Linux as one of its tasks

The Minimum Software Components of a Linux System (1)

- The Linux kernel needs a number of additional software components (it can't do much on its own)
- A mechanism to allow it to identify the hardware (processor, memory, peripherals...)
  - Device tree or data structure passed by the bootloader (ATAG...)
  - Static description of the platform compiled with the kernel
  - Automatic mechanism to identify devices (ACPI tables, enumeration of USB devices...)

The Minimum Software Components of a Linux System (2)

- A bootloader
  - Basic initialization of the platform (clock tree caches, DRAM controller...)
  - Loading the kernel from a mass storage medium storage media (hard disk, flash memory...)
  - Setting up the complementary structures in memory for the kernel startup (device tree, initial memory disk (initrd...))

The Minimum Software Components of a Linux System (3)

- A root file system

- Either a minimal system in RAM set up from an image provided to the kernel at (initrd or initramfs)

- Or a more complete file system mounted from a mass storage medium or from the network

- Note: in many systems the two methods above are used one after the other. Methods above are used one after the other (will be detailed in the following)

The Minimum Software Components of a Linux System (4)

- An init process (launched from /sbin/init)
  - First process executed by the kernel
  - Ancestor of all other processes
  - Once this process is launched, the kernel startup phase is over
  - It is this process that will be in charge of the rest of the system startup

Other Common Components

- Configuration files and executables for (SysV init or systemd)
- Shared libraries (at least libc)
- Shell and classic utilities (sh, mount, test...)
  - Most of the essential libraries and tools come from the GNU project, hence the name GNU/Linux
- Daemons (udev, sshd...)
- A console (keyboard/screen)

Construction of a Complete Linux System (1)

- Use of a Linux distribution (generic or specialized)
  - Debian
  - Ubuntu
  - Raspberry Pi OS (formerly Raspbian), based on Debian
  - ...
- Advantages : ready to use, maintenance (security) on the long term
- Disadvantages: difficult to adapt for a particular situation, difficult to optimize

Construction of a Complete Linux System (2)

- Linux distribution generator
  - Yocto Project : https://www.yoctoproject.org/
  - OpenEmbedded : http://www.openembedded.org
  - Buildroot : https://buildroot.org/
- Advantages: increased flexibility and optimization
- Disadvantages: more difficult to set up than a ready-made distribution, compilation time

Construction of a Complete Linux System (3)

- Creation of the different elements by hand
- Advantages: total flexibility, total understanding of the system
- Disadvantages: very long to set up, problems of dependency between tools or versions, difficult learning curve

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

References

[1] Aspencore. 2019 embedded markets study.
https://www.embedded.com/wp-content/uploads/2019/11/EETimes_Embedded_2019_Embedded_Markets_Study.pdf, March 2019.

[2] Embeddable linux kernel subset.
https://github.com/jbruchon/elks, 2020.

[3] GNU. General Public License, version 2.
https://www.gnu.org/licenses/old-licenses/gpl-2.0.html, June 1991.

[4] GNU. General public license, version 3.
https://www.gnu.org/licenses/gpl-3.0.html, June 2007.

[5] GNU. Lesser general public license, version 3.
https://www.gnu.org/licenses/lgpl-3.0.html, June 2007.

Outline

- **Administrative Introduction**
- **World of Embedded System**
- **Open Source Licences**
- **Linux and Embedded**
- **References**

Group Assignment

- Group of 3
- Build your file system
- Take help from internet
- Discuss in your group
- Support Material:
- https://opensource.com/article/19/4/create-filesystem-linux-partition

Group Assignment

- Group of 3
- Build/Compile your Linux Kernel
- Take help from internet
- Discuss in your group
- Support Material:
- https://phoenixnap.com/kb/build-linux-kernel